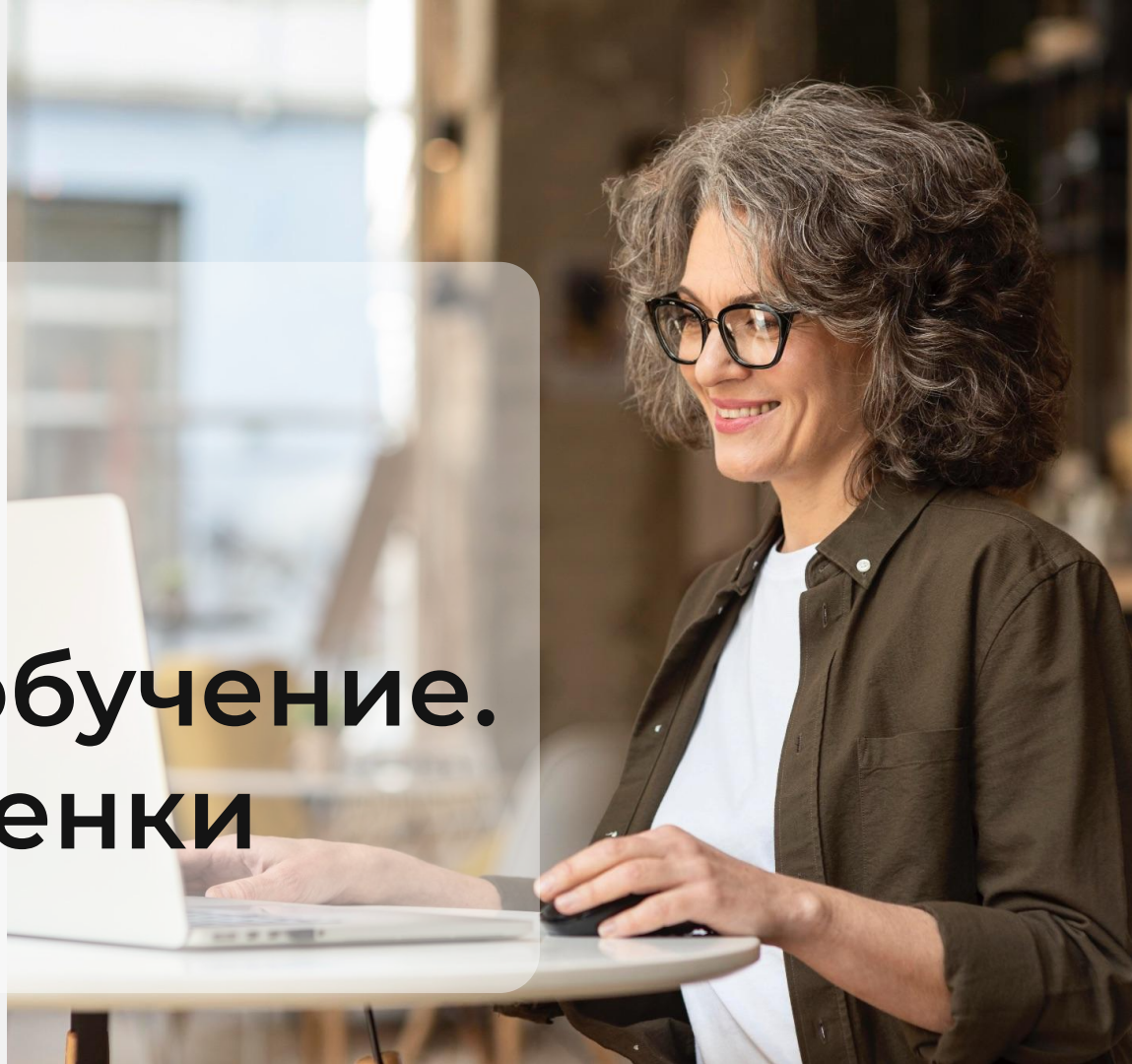


Python
for DA

Введение в машинное обучение. Метрики оценки модели



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  Линейная регрессия: Понятие и математические основы
-  Линейная регрессия: Реализация в Python
-  Логистическая регрессия: Понятие и математические основы
-  Логистическая регрессия: Реализация на Python
-  Деревья решений: Концепция и принцип работы
-  Деревья решений: Реализация на Python
-  Ближайшие соседи (K-Nearest Neighbors, KNN)
-  Концепция и принцип работы K-Nearest Neighbors (KNN): Реализация в Python

План занятия

- Введение в оценку моделей
- Accuracy: Определение и расчет
- Accuracy: реализация на Python
- Precision и Recall: Определения и расчеты
- Компромисс между показателями Precision и Recall
- Кривая Precision-Recall
- F1-Score: Определение и расчет



ОСНОВНОЙ БЛОК





Введение в оценку моделей



Оценка модели

Это шаг в рабочем процессе машинного обучения, который включает в себя оценку производительности обученной модели на отдельном наборе данных, который не использовался в процессе обучения.

Важность оценки эффективности модели

-  Обобщение на невидимые данные
-  Предотвращение избыточной и недостаточной подгонки: оверфиттинг и андерфиттинг
-  Выбор модели
-  Настройка гиперпараметров
-  Метрики производительности
-  Укрепление доверия и уверенности
-  Выявление слабых сторон модели
-  Обеспечение справедливости и выявление предвзятости

Предотвращение избыточной и недостаточной подгонки

Оверфиттинг

Возникает, когда модель слишком хорошо усваивает обучающие данные, включая шумы и выбросы, что приводит к низкой производительности на новых данных. Оценка модели на тестовом наборе помогает выявить оверфиттинг.

Андерфиттинг

Возникает, когда модель слишком проста, чтобы отразить основные закономерности в данных, что приводит к низкой производительности как на обучающих, так и на тестовых данных. Оценка модели помогает выявить андерфиттинг и подсказывает необходимость создания более сложной модели.

Метрики производительности

Меткость
(accuracy)

Доля правильно предсказанных
экземпляров от общего числа
экземпляров

Точность
(precision)

Recall
(чувствительность)

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Метрики производительности

Меткость
(accuracy)

Точность
(precision)

Recall
(чувствительность)

Доля верных положительных
предсказаний среди всех
положительных предсказаний

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Метрики производительности

Меткость
(accuracy)

Точность
(precision)

Recall
(чувствительность)

Доля истинно положительных
предсказаний от всех истинно
положительных

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Метрики производительности

Меткость
(accuracy)

Точность
(precision)

Recall
(чувствительность)

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Среднее гармоническое значение
между показателями precision и recall,
обеспечивающее баланс между ними

Метрики производительности

Меткость
(accuracy)

Точность
(precision)

Recall
(чувствительность)

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Среднее значение квадратичной
разницы между прогнозируемыми и
фактическими значениями

Метрики производительности

Меткость
(accuracy)

Точность
(precision)

Recall
(чувствительность)

F1-Score

Средняя квадратичная
ошибка (MSE)

R-квадрат
(R^2)

Мера того, насколько хорошо
предсказания модели соответствуют
фактическим значениям



ВОПРОСЫ





Accuracy: 

Определение
и расчет



Accuracy

Это метрика для оценки эффективности модели классификации. Измеряет долю правильно предсказанных экземпляров (как истинно положительных, так и истинно отрицательных) от общего числа экземпляров в наборе данных. Меткость говорит о том, как часто модель делает правильные прогнозы.

Формула для расчета точности



Accuracy = Количество правильных предсказаний/Общее количество предсказаний

Меткость в терминах компонентов матрицы ошибок

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN},$$

где:

- *TP* (True Positives): Количество экземпляров, правильно предсказанных как положительный класс.
- *TN* (True Negatives): Количество экземпляров, правильно предсказанных как отрицательный класс.
- *FP* (False Positives): Количество экземпляров, неверно предсказанных как положительный класс.
- *FN* (False Negatives): Количество экземпляров, неверно предсказанных как отрицательный класс.

Пример

	Предсказанное положительное	Предсказанное отрицательное
Фактическое положительное	50	10
Фактическое отрицательное	5	35

Пример

Из матрицы ошибок мы можем определить следующие значения:

$$TP=50, TN=35, FP=5, FN=10.$$

Используем формулу точности:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} = \frac{50+35}{50+35+5+10} = \frac{85}{100}$$

Меткость модели составляет 0,85, или 85%.

Accuracy



Преимущества

- Простота
- Интуитивная интерпретация

Ограничения

- Несбалансированные наборы данных
- Отсутствие детализации

Когда использовать Accuracy



Классы в наборе данных сбалансированы, то есть количество экземпляров в каждом классе примерно одинаково.



Стоимость ложноположительных и ложноотрицательных ошибок одинакова, и нет существенной разницы в значимости различных типов ошибок.



ВОПРОСЫ





Accuracy: реализация на Python

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
```

Шаг 2: Загрузка и подготовка набора данных

```
from sklearn.datasets import load_iris

# Загрузить набор данных
iris = load_iris()
X, y = iris.data, iris.target

# Преобразование в DataFrame для более удобного манипулирования
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = y
```

Шаг 3: Разбиение на обучающие и тестовые выборки

```
from sklearn.datasets import load_iris

# Загрузить набор данных
iris = load_iris()
X, y = iris.data, iris.target

# Преобразование в DataFrame для более удобного манипулирования
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = y
```

Шаг 4: Обучение модели логистической регрессии

```
# Инициализируем модель логистической регрессии
model = LogisticRegression(max_iter=200)

# Обучаем модель на обучающих данных
model.fit(X_train, y_train)
```

Шаг 5: Делаем предсказания

```
# Делаем предсказания на тестовом наборе  
y_pred = model.predict(X_test)
```

Шаг 6: Оценка модели

```
# Вычислить точность
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy}')
```

Шаг 7: Отображение матрицы ошибок

```
# Отображение матрицы ошибок
conf_matrix = confusion_matrix(y_test, y_pred)
print('Матрица ошибок:')
print(conf_matrix)

# Визуализация матрицы ошибок
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Прогноз')
plt.ylabel('Факт')
plt.title('Матрица ошибок')
plt.show()
```



ВОПРОСЫ





Precision и Recall:

Определения и расчеты



Precision (точность) и recall (чувствительность)

Это фундаментальные метрики, используемые для оценки эффективности моделей классификации, которые дают представление о способности модели правильно определять положительные экземпляры и особенно полезны в сценариях, где классы несбалансированы.

Precision



Определение

- Измеряет долю положительных предсказаний, которые действительно верны.
- Отвечает на вопрос: «Сколько из всех случаев, которые были предсказаны как положительные, были действительно положительными?»

Формула

$$Precision = \frac{TP}{TP+FP}$$

где:

- Истинные положительные результаты (TP): Количество экземпляров, правильно предсказанных как положительные.
- Ложноположительные результаты (FP): количество экземпляров, неверно предсказанных как положительные.

Пример

Модель, которая классифицирует электронные письма как спам или не спам.

Модель делает следующие прогнозы:

- Истинные положительные результаты (TP): 50 (письма, правильно идентифицированные как спам)
- Ложноположительные результаты (FP): 10 (письма, неправильно идентифицированные как спам)

Точность можно рассчитать следующим образом:

$$Precision = \frac{50}{50+10} \approx 0.83$$

83% писем, предсказанных как спам, на самом деле оказались спамом.

Recall



Определение

- Измеряет долю реальных положительных случаев, которые были правильно определены моделью.
- Отвечает на вопрос: «Сколько из всех фактических положительных примеров было правильно предсказано как положительные?»

Формула

$$Recall = \frac{TP}{TP+FN}$$

где:

- True Positives (TP): Количество экземпляров, правильно предсказанных как положительные.
- Ложные негативы (FN): Количество экземпляров, неверно предсказанных как отрицательные.

Пример

Классификация электронной почты. Модель делает следующие прогнозы:

- Истинно положительные (TP): 50 (письма, правильно идентифицированные как спам).
- Ложноотрицательные (FN): 20 (письма, неправильно идентифицированные как не спам).

Чувствительность можно рассчитать следующим образом:

$$Recall = \frac{50}{50+20} \approx 0.71$$

71% реальных спам-писем были правильно идентифицированы моделью.



ВОПРОСЫ





Компромисс между показателями Precision и Recall

Высокий precision, низкий recall

Сценарий

Модель очень консервативна в составлении положительных прогнозов. Она делает положительные прогнозы только тогда, когда очень уверена в этом

Результат

Количество ложноположительных результатов (FP) низкое, но количество ложноотрицательных результатов (FN) высокое

Пример

В системе обнаружения спама модель с высокой точностью будет редко отмечать легитимные письма как спам (низкая FP), но может пропустить много настоящих спамных писем (высокая FN)

Высокий recall, низкий precision

Сценарий

Модель очень либерально относится к положительным прогнозам. Она предсказывает положительные результаты даже при низкой уверенности

Результат

Число ложноотрицательных результатов (FN) низкое, но число ложноположительных результатов (FP) высокое

Пример

В системе медицинской диагностики модель с высоким коэффициентом отзыва будет отлавливать почти все реальные случаи заболевания (низкий FN), но она также может отмечать наличие заболевания у многих здоровых людей (высокий FP)



ВОПРОСЫ





Кривая Precision-Recall



Кривая «Precision-Recall»

Показывает соотношение precision (ось y) и recall (ось x) для различных пороговых значений. Помогает понять компромисс между precision и recall и выбрать оптимальное пороговое значение, исходя из конкретных требований задачи.

Ключевые моменты

Визуализация
компромисса

Площадь под кривой
(AUC)

Сравнение моделей

Пример



Кривая «Precision-Recall» на наборе данных Breast Cancer с алгоритмом Decision Tree.
Реализуем классификатор Decision Tree, чтобы предсказать, является ли опухоль рака молочной железы доброкачественной или злокачественной, используя набор данных Breast Cancer.
Построим кривую Precision-Recall, чтобы оценить эффективность модели.

Шаг 1: Импорт необходимых библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_recall_curve, precision_score, recall_score,
confusion_matrix, classification_report
from sklearn.datasets import load_breast_cancer
```

Шаг 2: Загрузка и подготовка набора данных

```
# Загрузить набор данных
data = load_breast_cancer()
X, y = data.data, data.target

# Преобразуем в DataFrame для более удобного манипулирования
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y
```

Шаг 3: Разделение на обучение и тестирование

```
# Разделите данные на обучающий и тестовый наборы  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

Шаг 4: Обучение модели дерева решений

```
# Инициализируем классификатор дерева решений с максимальной глубиной 3
model = DecisionTreeClassifier(max_depth=3, random_state=42)

# Обучите модель на обучающих данных
model.fit(X_train, y_train)
```

Шаг 5: Предсказания и расчеты Precision и Recall

```
# Делаем предсказания на тестовом наборе
y_pred = model.predict(X_test)
y_scores = model.predict_proba(X_test)[:, 1]

# Рассчитайте precision и recall
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
print(f'Precision: {precision}')
print(f'Recall: {recall}')

# Вывести отчет о классификации
print('Отчет о классификации:')
print(classification_report(y_test, y_pred))

# Вывести матрицу ошибок
conf_matrix = confusion_matrix(y_test, y_pred)
print('Матрица ошибок:')
print(conf_matrix)

# Визуализация матрицы ошибок
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=data.target_names, yticklabels=data.target_names)
plt.xlabel('Прогноз')
plt.ylabel('Факт')
plt.title('Матрица ошибок')
plt.show()
```

Шаг 6: Постройте кривую «Precision-Recall»

```
# Рассчитайте кривую Precision-Recall
precisions, recalls, thresholds = precision_recall_curve(y_test, y_scores)

# Построить график кривой Precision-Recall
plt.plot(recalls, precisions, marker='.')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Кривая Precision-Recall')
plt.show()
```



ВОПРОСЫ





F1-Score:

Определение и расчет



F1-score

Это метрика, используемая для оценки эффективности модели классификации. Представляет собой среднее гармоническое значение precision и recall, обеспечивая единый показатель, который уравнивает обе метрики.

F1-score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- Истинноположительные результаты (TP): Количество экземпляров, правильно предсказанных как положительные.
- Ложноположительные результаты (FP): количество экземпляров, неверно предсказанных как положительные.
- Ложноотрицательные результаты (FN): Количество экземпляров, неверно предсказанных как отрицательные.

Пример

	Предсказанное положительное	Предсказанное отрицательное
Фактическое положительное	50	10
Фактическое отрицательное	5	35

Пример

Из матрицы ошибок можно определить следующие значения:

- Истинные положительные результаты (TP): 50
- Ложные положительные результаты (FP): 5
- Ложноотрицательные (FN): 10

Используя эти значения, можно рассчитать точность и чувствительность:

$$Precision = \frac{50}{50+5} \approx 0.91, Recall = \frac{50}{50+10} \approx 0.83$$

Можно рассчитать F1-score:

$$F1 \approx 2 \times \frac{0.91 \times 0.83}{0.91 + 0.83} \approx 0.87$$

F1-score для этой модели составляет примерно 0.87.

F1-Score



Преимущества

- Баланс между precision и recall
- Полезен для несбалансированных наборов данных
- Нечувствительность к распределению классов

Ограничения

- Не различает типы ошибок
- Единая метрика



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



Задание

Примените алгоритм K-Nearest Neighbors (KNN) для классификации несбалансированного набора данных и вычисления F1-score. Мы будем использовать набор данных Breast Cancer из Scikit-learn, который является слегка несбалансированным.

Пошаговая реализация:

1. Импорт необходимых библиотек
2. Загрузите и подготовьте набор данных
3. Выполните разделение на обучение и тестирование
4. Обучение KNN-модели
5. Сделайте предсказания
6. Вычислите точность, чувствительность и F1-Score
7. Выведите матрицу ошибок



ВОПРОСЫ



Заключение

